

Tomcat and Web server

Lecture notes

Goal of the chapter

This chapter discusses about the basic concept of a web application using a Tomcat server.

1 Introducing Tomcat and Web server

Web is basically a system of Internet servers that supports formatted documents. The documents are formatted using a markup language called HTML (HyperText Markup Language) that supports links to other documents, like graphics, audio, and video files.

- When you type an address such as `www.deakin.edu.au/example_path/international-students.html` into the address field, your browser first resolves `www.deakin.edu.au` (i.e., the name of the web server) into the corresponding Internet Protocol (IP) address, usually by asking the Domain Name Server provided by your Internet service provider (ISP). Then your browser sends an HTTP (HyperText Transfer Protocol) request to the newly found IP address to receive the content of the file identified by `/example_path/international-students.html`.
- When a web server receives that request, it first determines whether the resource being requested is static or dynamic in nature. A static resource is one that is identical no matter how many times, or in what order, that resource is requested. In reply, the web server sends an HTTP response containing a plain-text HTML (HyperText Markup Language) page for static resources. Images and other non-textual components, such as sound and video clips, only appear in the page as hyperlinks to those resources stored either on the same server or another server on the Internet.
- The browser receives the response, interprets the HTML code contained in the page, requests the non-textual components from the server, and displays the content. This has been displayed in Figure 1

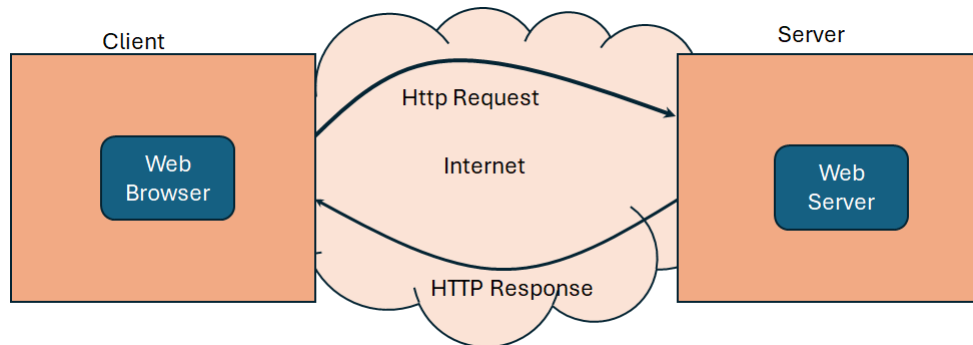


Figure 1: A Web server with static resources requests.

However for dynamic resources,¹ Web servers require an additional software component that is registered with the web server, called a servlet container on the Java EE platform. In the Internet Information Server (IIS) world, access to this servlet container is offered by implementing an Internet Server Application Programming Interface (ISAPI) extension, which is a Dynamic Link Library (DLL) that allows you to extend the functionality of IIS.

When a request comes in that is directed at a dynamic resource, the web server simply hands off that request to its associated servlet container (Tomcat). The container then springs into action. It checks whether it is a servlet request or Java server page (JSP) request. If it is a servlet, then it sends directly to servlet engine. The servlet engine component invokes the appropriate servlet that represents the requested dynamic resource. This servlet is responsible for generating the response. The generated response is returned by the servlet container to the web server which, in turn, returns it to the requesting user. This interaction is depicted in the following figure [2](#).

1.1 Servlet container

A servlet container, that is, an environment within which servlets can live and prosper. A container provides a lot of administrative support to servlets, allowing programmers to focus on the core application logic that is to be implemented, without having to bother about low level specifics such as session management and class loading. Tomcat is classified as a servlet container.² It has components such as servlet engine, JSP engine, The HTTP Connector and so on. The components may vary in different version.³

¹References: Book: Tomcat 6 Developer's Guide, Damodar Chetty

²References: Book: Tomcat 6 Developer's Guide, Damodar Chetty

³<https://tomcat.apache.org/tomcat-5.5-doc/architecture/overview.html>

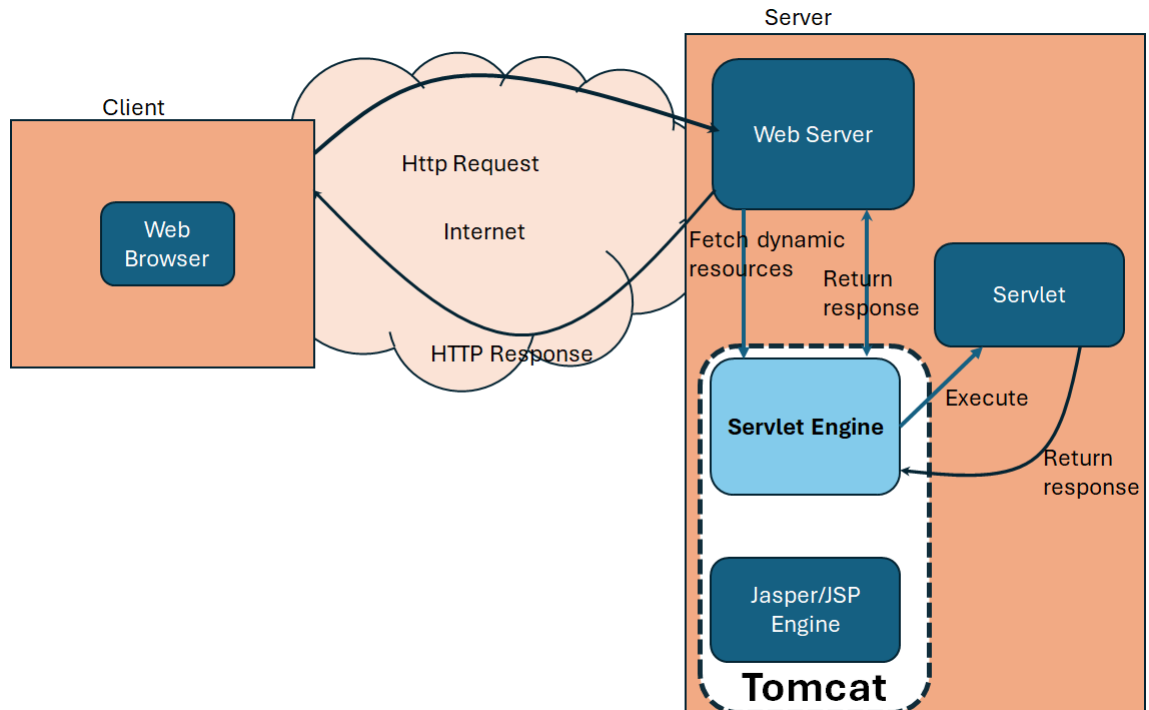


Figure 2: A Web server with dynamic resources requests (Introducing Tomcat).

1.2 Java server page (JSP) as a dynamic resource request

JSP is a technology that lets you add dynamic content to web pages. In absence of JSP, to update the appearance or the content of plain static HTML pages, you always have to do it by hand. Even if all you want to do is change a date or a picture, you must edit the HTML file and type in your modifications. With JSP, you can make the content dependent on many factors, including the time of the day, the information provided by the user, the user's history of interaction with your website, and even the user's browser type. This capability is essential to provide online services in which you can tailor each response to the viewer who made the request, depending on the viewer's preferences and requirements. A crucial aspect of providing meaningful online services is for the system to be able to remember data associated with the service and its users. That's why databases play an essential role in dynamic web pages. But let's take it one step at a time.

- If the request is for a JSP page, servlet container forwards this to its component "JSP engine". The JSP engine loads the JSP page from disk and converts it into a Java servlet. From this point on, this servlet is

indistinguishable from any other servlet developed directly in Java rather than JSP,

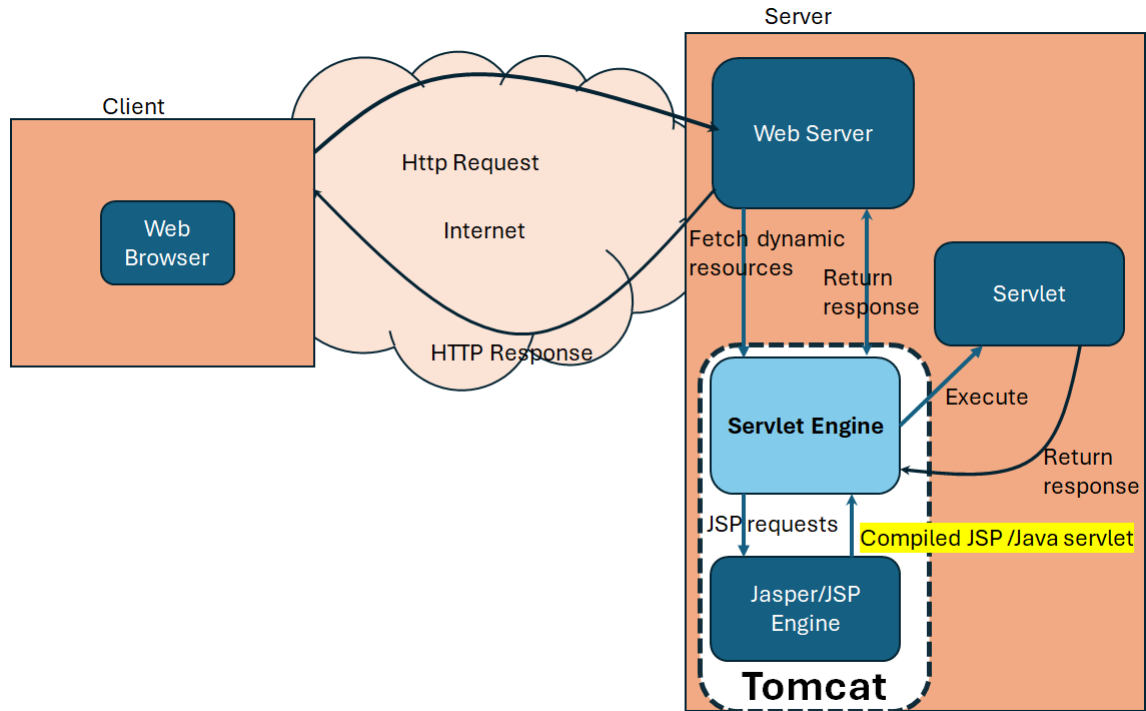


Figure 3: A Web server with dynamic resources (JSP requests).

- The JSP engine compiles the servlet into an executable class and forwards the original request to the servlet engine. Note that the JSP engine only converts the JSP page to Java and recompiles the servlet if it finds that the JSP page has changed since the last request. This makes the process more efficient than with other scripting languages (such as PHP) and therefore faster. ⁴
- The servlet engine loads the servlet class and executes it. During execution, the servlet produces an output in HTML format, which the servlet engine passes to the web server inside an HTTP response.
- The web server forwards the HTTP response to the browser.
- The web browser handles the dynamically generated HTML page inside the HTTP response exactly as if it were a static page. In fact, static and

⁴Reference Book: Beginning Jakarta EE Web Development Using JSP, JSF, MySQL, and Apache Tomcat for Building Java Web Applications, Authors: Luciano Manelli, Giulio Zambon

dynamic web pages are in the same format. This has been depicted in Figure 3.

1.3 Our first web application based on servlets

Goal

We would like a web application that allows a user to interact with a fitness coach by asking him/her a daily workout. So these are the steps:

- The user access our server by using the correct URL.
- The type of the workout will be determined by the server.

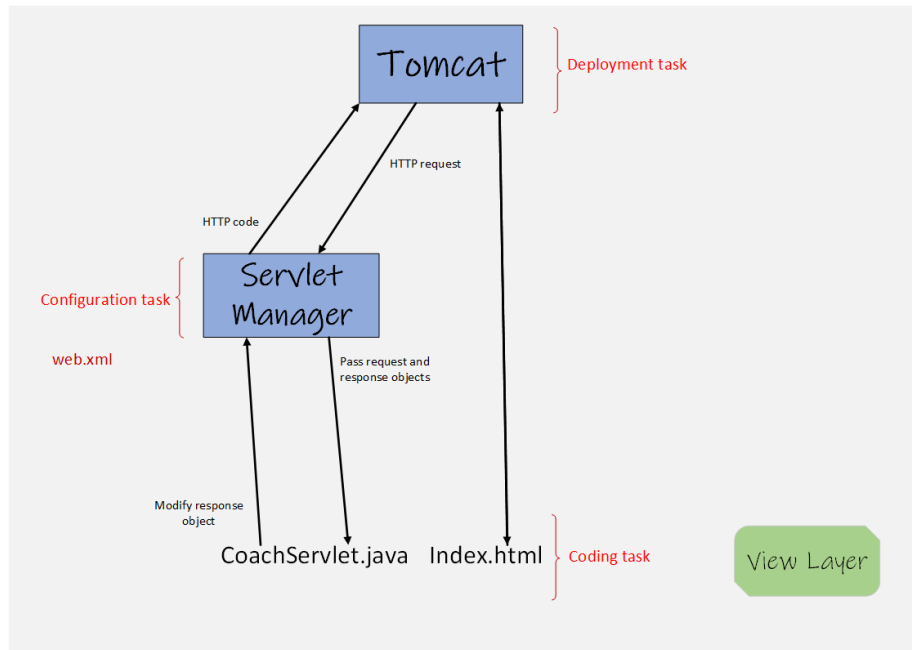


Figure 4: Architecture of our servlet-based web app.

We are going to deploy our web application based on servlets within the Tomcat server. Tomcat requires web applications to be deployed within a particular format. The reason being that web applications can become fairly complex hence a bit of structure and organisation helps. Rather than learning the details of that format, we are going to create a web project that can be directly deployed into tomcat. These are the steps.

1.3.1 Creating a Dynamic Web project in Eclipse

- In Eclipse, create a new web project by going to New > Dynamic Web Project. Click next, click on “generate content descriptor”, click finish.
- Let us say that you called this project “coachwebapp”

1.3.2 Coding the static HTML page and a Servlet class

- Right click on the folder src/main/webapp, go to new/HTML file, and create a new html file called index.html with the following HTML code:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="ISO-8859-1">
5 <title>SIT218/SIT738 Secure Coding =>
6   servlet interface example</title>
7 </head>
8 <body>
9 <h1>SIT218 Secure Coding</h1>
10 <br>
11 <br> *****
12 <br>
13 <br>
14 <h3>
15   <a href="workout">
16     Click here to get a workout...</a>
17 </h3>
18 <br>
19 <br> *****
20 <br>
21 <br>
22 </body>
23 </html>
24
```

- It is worth noting that the <a> HTML element (or anchor element), with its href attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address. In this case, href="workout" tells the browser that the Get request created once the link is clicked should be directed to the URL created by appending /workout to the current URL.
- The next step is to create a Java class (a Servlet) that will process the Get request /workout. We do so by right clicking again on the project, and telling Eclipse to create a Java class.
 - For this example use the package edu.deakin.sit218.coachwebapp
 - Use the class name CoachServlet
 - Click finish.
 - Make your class extends the class HttpServlet

- Implement the method service. Your code should look as follows:

```

1 package edu.deakin.sit218.coachwebapp;
2
3 import java.io.IOException;
4 import java.io.PrintWriter;
5 import java.util.Random;
6
7 import javax.servlet.ServletException;
8 import javax.servlet.http.HttpServlet;
9 import javax.servlet.http.HttpServletRequest;
10 import javax.servlet.http.HttpServletResponse;
11
12 public class CoachServlet extends HttpServlet {
13     private static final long serialVersionUID = 1L;
14
15     public CoachServlet() {
16         super();
17     }
18
19     protected void service(HttpServletRequest request,
20                           HttpServletResponse
21                           response) throws ServletException, IOException {
22         //Here goes your code. We will get there!
23     }
24 }
25
26 }
27
28

```

- Now let's put some code inside the service method.

```

1 PrintWriter out = response.getWriter();
2 Random r = new Random();
3 if (r.nextBoolean())
4     out.println("<HTML> <h1> No training today "
5         + "</h1> </HTML>");
6 else
7     out.println("<HTML> <h1> Spend 30 min running "
8         + "</h1> </HTML>");

```

- This coach implementation is essentially tossing a coin and telling us to run if it turns head, or chill otherwise.
- Our Java code is ready. All is left is to tell the server that CoachServlet is a servlet and that it should be called if the URL pattern '/workout' is seen in a request. We are going to use deployment descriptor file web.xml for that. If you haven't done so, right click on your project, then go to Java EE Tools and click on Generate Deployment Descriptor Stub. This will generate a web.xml file within src/main/webapp/WEB-INF/. Open that file and add the following configuration.

```

1  <servlet>
2    <servlet-name>coachervlet</servlet-name>
3    <servlet-class>edu.deakin.sit218.coachwebapp.CoachServlet</
4      servlet-class>
5    </servlet>
6  <servlet-mapping>
7    <servlet-name>coachervlet</servlet-name>
8    <url-pattern>/workout</url-pattern>
9    </servlet-mapping>
10

```

The first entry, under the root servlet element in web.xml, defines a name for the servlet and specifies the compiled class that executes the servlet. (Or, instead of specifying a servlet class, you can specify a JSP.) The servlet element also contains definitions for initialization attributes and security roles for the servlet.

The second entry in web.xml, under the servlet-mapping element, defines the URL pattern that calls this servlet.

Annotations

The Java annotation `@WebServlet("/workout")` can be used to specify that a class is a Web Servlet with a `"/workout"` URL pattern. Java annotations are short and convenient, while XML configuration is long and verbose. In this course we are using both, annotations and XML, to better understand what's going on under the hood.

- Now run your project within Eclipse: Run As/Run on Server. This will make your web app run within Eclipse.